

An Exact and Interpretable Solution to Wordle

Dimitris Bertsimas and Alex Paskov

Jeremy Rosen and Shahzoor Safdar

Georgetown University

November 30, 2022

Contents

- 1 Introduction
- 2 Dynamic Programming
- 3 Implementation
- 4 Decision Trees
- 5 Discussion

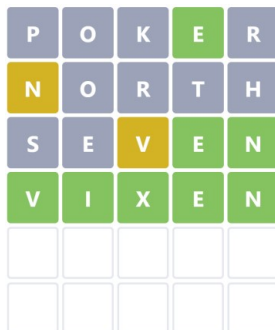
Introduction

Overview

- Authors use “Exact Dynamic Programming” to solve Wordle.
We break down their contributions as follows:
 - Define a finite-state Markov Decision Process for Wordle and construct a tractable dynamic programming algorithm to solve it
 - Produce Decision Trees that help humans play Wordle better
 - Discuss why prior approaches using other methods have failed

Rules of Wordle

- Goal is to guess a five-letter word in up to six guesses
- After each guess, each letter is colored green, yellow, or grey
 - **Green:** Your letter is in the word in the correct position
 - **Yellow:** Your letter is in the word in the wrong position
 - **Grey:** Your letter is not in the word
- If your guess has a letter k times and the word has it $n < k$ times, the last $k - n$ tiles are colored grey (indicating the letter doesn't appear $> n$ times)



Optimal first guess = SALET

“A light round helmet extending over the back of the neck” (Collins Dictionary)



Dynamic Programming

State and Control

- **State:** s_t is set of all possible words that can still be a solution (Markovian since next state only depends on current state and action)
 - Suppose $s_4 = \{\text{AWAKE, BLUSH, FOCAL}\}$
 - We guess THETA and see 
 - Then $s_5 = \{\text{FOCAL}\}$ since E and H aren't in word
 - Note that $|s_0| = 2,315$ since there are 2,315 possible solutions
- **Control:** Action is word in set of all valid guesses A . Note that $|A| = 10,657$, i.e. not all guesses can be a solution

Cost and Transition

- **Cost Function:** $c_t(s, a) = 1, \forall t \leq 5$ and $c_6(s) = \infty$
 - If policy has finite cost, it wins the game
 - Minimizing c is equivalent to minimizing expected number of guesses
- **Transition Function:** Given s_t and a , let $P(s_t, a)$ = collection of all states we can transition into. Note that $|P(s_t, a)| \leq 3^5$ (3 possible colors, 5 letters)
 - Fix s_t and a . For each $w \in s_t$, suppose it's the solution and record s_{t+1} .
 $P(s_t, a)$ is the union of all the s_{t+1} 's we get across the w 's
 - To get transition probability $p_t(s_{t+1}; s_t, a)$ for any $s_{t+1} \in P(s_t, a)$,
 count the number of times s_{t+1} appears across the w 's and divide it by $|s_t|$

Bellman's Equation

$$V_t^*(s_t) = \begin{cases} 1, & \text{if } t < 6, |s_t| = 1 \\ \min_{a \in A} \left\{ 1 + \sum_{s_{t+1} \in P(s_t, a)} p_t(s_{t+1}; s_t, a) V_{t+1}^*(s_{t+1}) \right\}, & \text{if } t < 6, |s_t| > 1 \\ \infty, & \text{if } t = 6 \end{cases}$$

- Note that at $t = 0$, you make your first guess. And if $t = 6$, you've lost
- Solve equation for $V_0^*(s_0)$ with recursion by searching through all $a \in A$ and recursively solving $V_{t+1}^*(s_{t+1})$ for the states we can transition into

Implementation

What is Recursion?

- In programming, recursive functions are functions that call themselves
- *"To understand recursion, you must first understand recursion."*
- **Example:** Computing factorials!
 - Recall that $n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 2 \cdot 1$
- We can define factorials recursively:

$$n! = \begin{cases} n \cdot (n - 1)! & \forall n > 1 \\ 1 & n = 1 \end{cases}$$

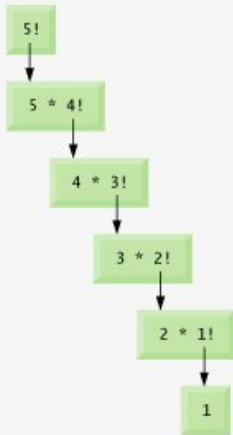
Implementing Recursion

```
function out = factorial(n)
    % computing factorials recursively

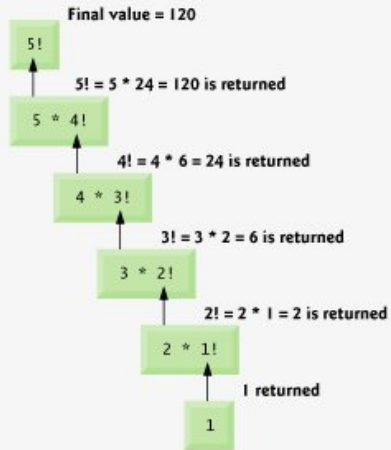
    if n == 1
        out = 1; % base case
    else
        out = n * factorial(n-1); % recursive call
    end

end
```

A Look Under the Hood (Deitel 2004)



(a) Sequence of recursive calls.



(b) Values returned from each recursive call.

Recursion and Dynamic Programming

- Clear similarities between the recursive definition of factorials and the value function of a finite-horizon dynamic programming problem
 - ① Both make recursive calls
 - **Factorial:** To compute $n!$, you must first compute $(n - 1)!$
 - **Value Function:** To compute $V_t^*(s_t)$, you must first compute $V_{t+1}^*(s_{t+1})$
 - ② Both have “base” cases, or cases where the value does not depend on a time-stepped or shifted iteration of the function itself
 - **Factorial:** $n! = 1$ for $n = 1$
 - **Value Function:** $V_t^*(s_t) = \infty$ for $t = 6$, and $V_t^*(s_t) = 1$ for $t < 6$, $|s_t| = 1$

Their Base Cases

Algorithm 2 Compute $V_t^*(s_t)$

Input: Integer t , State s_t , Dictionary[(Set,Integer)→Float] V_{memory}

Output: Float $V_t^*(s_t)$

if $t = 6$ or $(t = 5$ and $|s_t| > 1)$ **then** ▷ Optimization 1

return ∞

else if $t = 5$ **then**

return 1

else if $|s_t| = 1$ **then**

return 1

else if $|s_t| = 2$ **then** ▷ Optimization 2

return 1.5

else if $(s_t, t) \in V_{memory}$ **then**

return $V_{memory}[(s_t, t)]$

end if

Their Recursive Steps

```

state_value  $\leftarrow \infty$ 
for action  $\in (s_t \cup (A - s_t))$  do                                ▷ Optimization 5
    temp, next_states  $\leftarrow 1, \text{Get\_Transition\_Info}(s_t, \text{action})$     ▷ Optimization 3
    if next_states.size = 1 and  $s_t \in \text{next\_states}$  then
        continue                                                    ▷ Skip action, no info revealed
    end if
    for  $s_{t+1} \in \text{next\_states}$  do
        temp  $\leftarrow \text{temp} + \frac{2|s_{t+1}|-1}{|s_{t+1}|}$                                 ▷ Optimization 4
    end for
    for  $s_{t+1} \in \text{next\_states}$  do
        if temp  $\geq \text{state\_value}$  then
            break
        else if  $s_{t+1} = \{\text{action}\}$  then
            continue
        end if
        temp  $\leftarrow \text{temp} + V_{t+1}^*(s_{t+1})$ 
    end for
    if temp < state_value then
        state_value  $\leftarrow \text{temp}$ 
    end if
end for
return state_value

```

Hacks, Part 1

- They come up with 5 “hacks” to speed up the runtime
- **Hack 1:** Reducing the time-horizon from 6 periods to 5:
 - In $t = 5$, if you know (don't know) the word, your expected cost is 1 (∞)
- **Hack 2:** When $|s_t| = 2$ and $t < 5$, $V_t^*(s_t) = 1.5$
 - You get it right and win, or you get it wrong and need one more guess to win
- **Hack 3:** Efficiently calculating transition probabilities
 - Given a guess and coloring, it's easy to check if any solution is still feasible
 - So they “pre-compute this boolean” to find s_{t+1} for each $w \in s_t$ (see Slide 9)

Hacks, Part 2

- **Hack 4:** Pruning sub-optimal guesses when searching through $a \in A$:
 - For any state s_t , compute a lower bound on $V_t^*(s_t)$
 - For some guess a , apply lower bound to $V_{t+1}^*(s_{t+1})$ for each $s_{t+1} \in P(s_t, a)$
 - As we solve each $V_{t+1}^*(s_{t+1})$, update summation in BE with exact values
 - If another guess ever beats guess a 's lower bound, then prune guess a
- **Hack 5:** When searching through guesses, start with those that are possible solutions because the optimal guess a^* is usually one of them

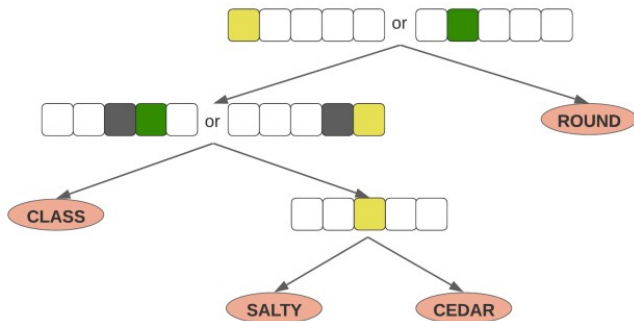
Runtime

- Program takes days to run, even with C++ and parallelization over 64 cores
- Increasing vocabulary size wouldn't be too bad because of Hacks 3–5
- But if Wordle had 7 rounds and they didn't explicitly constrain the policy function to solve the game in 5 or 6 rounds, the runtime would blow up

Decision Trees

OCT-H

- They use Optimal Classification Trees with Hyperplanes (OCT-H) to display optimal policies for humans to understand
- To use a tree, first make a guess and observe your coloring
- Then, starting from the top of the tree, if the answer to the question at any node is “Yes,” go down and right. If it’s “No,” go down and left
- Continue going down until you reach a leaf conveying the optimal next guess



Optimal First Guess

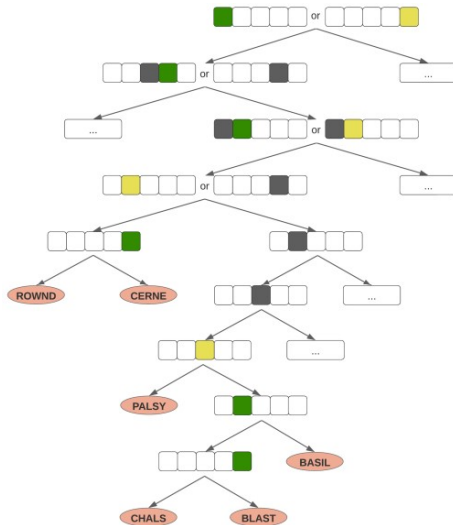
- Recall the optimal first guess is SALET
 - If we assign equal probability to each of the 2,135 potential solutions being the solution, it yields an expected number of guesses of 3.421
 - And it finds the hidden word in no more than 5 guesses
- AUDIO has the most vowels, but it isn't as good because it lacks E, as well as commonly-seen consonants like T
- Other good guesses are REAST (“(esp of horses) to be noisily uncooperative,” Collins Dictionary), TRACE, CRATE, and SLATE

Guess 2 Decision Table

→courd	→groin	→north	→rownd
→cerne	→mohur	→drone	→trunic
→trite	→fugly	→pilot	→tulip
→idler	→filet	→himbo	→tilde
→courd	→clung	→troll	→bevor
→fleet	→hotel	→doing	→excult
→title	→corny	→thing	→pubco
→gormy	→facet	→rehem	→crumb
→bathe	→rally	→tally	→paler
→valet	→valve	→bingy	→fault
→natal	→ghyll	→later	→cadge
→lathe	→brond	→croup	→train
→anger	→after	→beard	→cedar
→grate	→allay	→allot	→alley
→relax	→eclat	→delta	→corni
→fling	→trial	→alien	→alter
→gnarl	→pecan	→plate	→poncy
→churn	→pinko	→hewer	→sweet
→steed	→prink	→swept	→minke
→silky	→split	→solve	→plink
→spilt	→stool	→spiel	→sleet
→steel	→chemo	→smelt	→stole
→copsy	→saint	→satyr	→safer
→saucy	→saute	→aldol	→salty
→salve	→sadly	→cramp	→start
→prink	→pharm	→sweat	→grike
→solar	→splat	→child	→shalt
→stalk	→shale	→stale	→muchc
→doric	→durgy	→risen	→unset
→ester	→rhone	→quich	→troth
→islet	→welsh	→floss	→lusty
→loser	→flesh	→hings	→waist
→attap	→pause	→cowps	→palsy
→false	→basil	→ensel	→lapse
→crags	→acerb	→trash	→askew
→asset	→abuse	→yeast	→tcase
→chals	→blast	→leash	→least

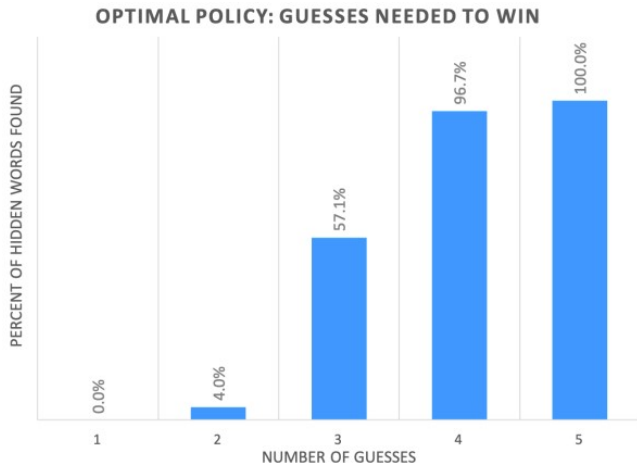
(Guess 1 was SALET)

Guess 2 Decision Tree



Optimal Policy

- By 2 guesses, it wins 4.0% of the time. But by 3, it wins 57.1% of the time
- It never needs the 6th guess (other algorithms may not even get it in 6)



Discussion

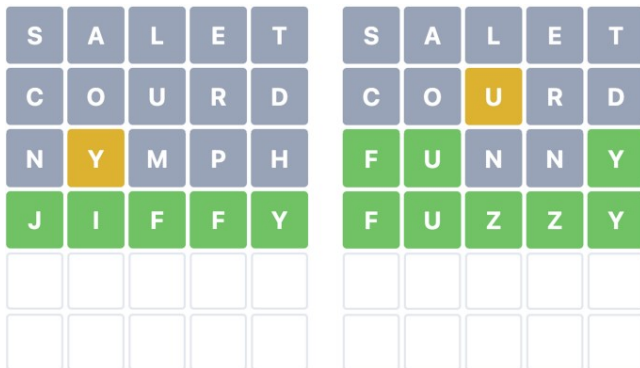
Why Algorithm Beats Humans

- Suppose hidden word is EIGHT, and your first guess is LIGHT
- You can search over TIGHT, RIGHT, etc. and not get it in 6 guesses
- Algorithm guesses FERMS (rents for a farm) to rule out all but EIGHT



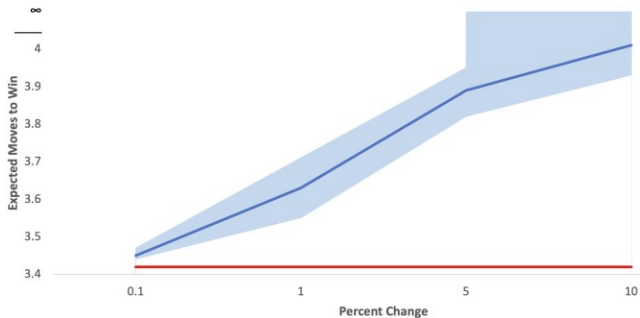
What if you get all greys?

- Suppose algorithm guesses SALET and gets an all-grey coloring
- Algorithm then guesses COURD (covered) to rule out more letters
- All-grey isn't so bad; it tells you a lot about what's not in the word!



Why Algorithm Beats Previous Approaches

- Previous approaches use approximations to make computation easier
- But they don't converge on same policy, and sometimes they even lose
- **Experiment:** Add noise to $V_3(s_3)$ of $x\%$ of its value and then compute optimal policy. Do this 25 times since noise is random
- **Result:** The more noise added, the greater the expected number of guesses needed to win. If $x = 10$, sometimes the algorithm loses



Conclusion

- Dynamic programming solves Wordle!
- Decision Trees let humans interpret the optimal policy
- Previous approximation-based approaches failed because the optimal policy is sensitive to small changes in the value function



Wordle



WordleOpt